DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Data in R

CMSC 124 Problem Set 2

Prepared by: Rene Andre Bedonia Jocsing

Published: October 13, 2026 **Deadline:** October 21, 2026

Suppose you run this expression:

```
c(TRUE, 2L, 3.5)
```

The result is a vector containing `1`, `2`, and `3.5`. That answer hides the part R decided for you. The logical value became a number, the integer became a double, and the result kept length 3 without gaining dimensions. One line crossed the data-type questions from Unit 5 and the coercion questions from Unit 6.

Last week you built ten data types by hand in C, where a value becomes another type only because you wrote the code that converts it. R is the other extreme. Types belong to values at runtime. A name can later refer to a value of another type. R also converts values on its own. Sebesta calls an implicit type conversion **coercion**. This problem set uses his term **mixed-mode assignment** for subassignment that widens an atomic vector so a new value fits.

This problem set makes those decisions visible. You'll predict 16 expressions before you run them. Each prediction has four parts: value, `typeof`, length, and dimensions. Then you'll write the same score transformation twice. One version uses R's vector operations. The other visits one element at a time. Their outputs must agree, but they get there through different evaluation models.

This is a paired, asynchronous assignment with seven full workdays. It uses only base R, so package installation can't become the assignment by accident. Most of you meet R for the first time here, so Part 1 also points to the guide you're expected to read while you work. This manual carries the rules the cases grade. It doesn't replace the introduction to R in the required reading.

A Small R Syntax Map

The opening expression already uses two R conventions. `c()` combines its arguments into a vector. The `L` suffix makes `2L` an integer, while a bare `2` is a double. R has no separate scalar data type. This problem set calls an operation scalar when it works on one element at a time. Each extracted element is still a length-one vector.

R usually writes assignment with `<-`. In `x <- 1:3`, R evaluates the right side and binds the result to `x`. The `:` operator builds the integer sequence `1, 2, 3`. Braces group several expressions into one block. R evaluates the block from top to bottom. Its last expression supplies the block's value.

The case corpus wraps each expression in `quote()`. That function stores an unevaluated call as a language object, which lets the runner choose and evaluate one case later. It doesn't run the call when `cases.R` loads.

Labels inside `c()` become the vector's `names` attribute:

```r
x <- c(first = 8L, second = 5L, third = 9L)
x[c(3L, 1L)]
```

The index requests positions 3 and 1 in that order. The selected values keep the attached names in the same order.

Why Four Predictions?

In R, the printed value is only one part of an object's behavior.

```r
data.frame(id = 1:2, passed = c(TRUE, FALSE))
```

This prints like a table. Underneath, `typeof()` reports `"list"`, `length()` reports 2, and `dim()` reports two rows by two columns. The four observations answer different questions:

Field	Question
`value`	What values does the expression produce, and in what order?
`type`	What storage mode does `typeof()` report?
`length`	How many top-level elements does `length()` count?
`dim`	Does the object carry dimensions, and what are they?

A matrix drives the distinction home. A 2 by 3 matrix has length 6 because six values live in it. Its dimensions are 2x3 because R also remembers how to arrange them. A plain vector of those six values has the same length and no dimensions.

`typeof()` ***Is Not*** `class()`

The type field asks what the object is made of. The class field asks how the object acts. R answers those questions with two functions that often disagree.

```r
x <- data.frame(id = 1:2, passed = c(TRUE, FALSE))
typeof(x)   # "list"
class(x)    # "data.frame"
```

A data frame acts like a table, so `class()` says `data.frame`. It's built out of a list of columns, so `typeof()` says `list`. A factor tells the same story in different words. It acts like a set of labels but stores integers, so `class()` says `factor` while `typeof()` says `integer`:

```r
y <- factor(c("red", "blue", "red"))
typeof(y)   # "integer"
class(y)    # "factor"
```

Nothing about `y` is a character value. The labels you see printed are the level names, and what the vector holds is the codes that point at them.

Predict what `typeof()` reports. Every case in this set asks for storage, so when your instinct hands you the word you'd see in a printout, it's probably handing you the class.

The grader uses a compact spelling for values. Square brackets hold an atomic vector, `matrix[2x3](...)` includes the dimensions, and `list{...}` shows the top-level items of a list. Part 1 gives the complete notation with an example of every form. This is grader notation. It isn't R syntax.

Four fields also mean four reasons a prediction can miss. A wrong `value` often points at one rule and a wrong `type` at another, so four fields turn one guess into a short diagnosis. `REASONING.md`, which Part 1 introduces, is where you write that diagnosis down before you run anything. Predicting the value alone would hide which rule decided the storage.

Learning Objectives

By the end of the week, you should be able to:

- Predict how R coerces values inside an atomic vector, including which type wins when implicit type conversion has a choice
- Trace element-wise operations when operand lengths differ
- Select vector elements by positive, negative, logical, or zero indices
- Explain how factors use integer storage plus level labels
- Distinguish atomic vectors, lists, matrices, arrays, and data frames by their observable structure
- Keep missing values from contaminating a computation
- Use vectorized expressions and subassignment to transform a data set, and recognize a mixed-mode assignment when one widens a vector under you
- Compare a vectorized implementation with a scalar loop over the same input

---

The Pair

I assign pairs at random across every section. You don't choose your partner. You probably won't work with your laboratory partner. If the pooled roster has an odd number of students, one trio receives the same work.

The published pairing sheet is authoritative. Both members predict every case before comparing answers. Splitting the table in half would defeat the part that asks you to reason about all 16 expressions. You may divide the four functions after that, but review each other's implementation before the final run.

Your schedules can differ. This is asynchronous work, so the repository is where you meet. Push small commits, say what you did in the message, and open an issue on the fork as soon as you're stuck. If your partner goes quiet for a couple of days, or the two of you can't find any overlapping time, tell me while there's still most of a week left. The commit history already records who did what, so you don't need to build a case first. What I can't do is fix it the night before the deadline.

This belongs to the lecture component. The laboratory booking, progress-report, and defense rules don't apply. The lecture deadline is strict.

R Setup

You need R 4.3 or newer and Git. The required reading and the supplied workflow both use R 4.6.1, the version the published results come from. Anything at or above the floor runs the same checks. The project uses base R only. It runs everything through `Rscript`, the command-line program that runs an R file without opening the R graphical interface. No R packages are required.

Windows

Install R and Git from PowerShell:

```
winget install --id RProject.R --exact
winget install --id Git.Git --exact
```

Git for Windows brings **Git Bash**, a terminal that understands the Unix commands this manual uses. Open it from the Start menu and run every later command from there. The two `winget` lines above are the only ones that belong in PowerShell. PowerShell has no `chmod` and won't run `./check.sh`, so a command pasted into the wrong window fails in a way that looks like a broken starter.

Winget installs R without putting it on `PATH`, so `Rscript` isn't found until you add R's `bin` directory once, in PowerShell:

```
[Environment]::SetEnvironmentVariable('Path', [Environment]::GetEnvironmentVariable('Path',
  ↪  'User') + ';C:\Program Files\R\R-4.6.1\bin\x64', 'User')
```

The path points to the version you installed. Winget installs the current one. This manual and the supplied workflow pin R 4.6.1. Close Git Bash and open it again, and `Rscript --version` should work.

Ubuntu

The Ubuntu repository carries R. Install it with:

```
sudo apt update
sudo apt install -y r-base
```

macOS

Install R with Homebrew:

```
brew install --cask r
```

Verify the Runtime

Run:

```
Rscript --version
```

The first line should identify R 4.3 or newer. If the command isn't found, fix the installation before cloning the starter. Every project command calls `Rscript`.

Get the Starter

Fork cmsc-124-ps2-starter and clone your fork. A fork is a snapshot: a fix I push to the starter mid-week never reaches it, so what you were graded against can't change under you.

```
git clone https://github.com/<your-account>/cmsc-124-ps2-starter.git
cd cmsc-124-ps2-starter
chmod +x run check.sh lint
./check.sh
```

One of you forks the repository under your account, keeps the fork public, and adds the other member as a collaborator, since the fork is the pair's deliverable. Then open the Actions tab on the fork and enable workflows. GitHub disables them on every fork. At first, that tab shows a short notice with an enable button. GitHub documents the rule and the fix under Workflows in forked repositories. The workflow run is a deliverable, so don't discover this on Day 7.

A fresh starter reports `1/76 checks passed` and exits 1. That red run is the correct starting state. Sixty-four checks belong to the prediction table. Ten belong to the functions in `src/analysis.R`, including two checks that keep the vector and scalar implementations on their required evaluation paths. One belongs to `ANALYSIS.md`, so it fails while the placeholder is there and stays failed until the file lands inside the 300 to 450 word range. One belongs to `REASONING.md`, so it fails until every case carries an entry. The one starting pass confirms that the vector stub doesn't contain a forbidden loop.

The Repository

```
cases/cases.R            16 quoted expressions, P01 through P16
predictions.tsv          your four predictions for every expression
REASONING.md             the rule behind every case, written before you run
src/analysis.R           four function stubs
tests/expected.tsv       every published expected result
tests/check_all.R        the complete public grader
scripts/support.R        canonical value descriptions
scripts/run_case.R       the one-case runner
scripts/validate_predictions.R  the prediction table's form checks
scripts/lint_predictions.R  the report that ./lint prints
run  lint  check.sh      the course run contract
ANALYSIS.md              the final comparison you write together
.github/workflows/test.yml  your CI run of the same checks
```

You edit five files: `predictions.tsv`, `REASONING.md`, `src/analysis.R`, `ANALYSIS.md`, and `README.md`, the last one replacing the starter's own. Keep the case corpus, expected table, runner, grader, and the `scripts/` helpers unchanged. The grader evaluates all 16 expressions before scoring and stops if R disagrees with the published expected table.

`check.sh` is the complete public automated check. The same script runs on your machine, in your GitHub Actions workflow, and during grading. The repository contains every automated check, with no hidden suite. The rubric separately assesses the analysis, the prediction history, collaboration, and submitted workflow runs. Grading starts from the starter tag and reads the fork's diff against it, so an edited fixture is visible in the open.

That policy creates an awkward fact for a prediction assignment. The public file `tests/expected.tsv` contains the answers because grading can't depend on secret results. Don't open it until your prediction commit exists. Git records when your table changed. It can't prove which files you read first, so this part depends on your honesty.

Part 1: Predict Before Execution

Open `cases/cases.R` and `predictions.tsv` side by side. The prediction file starts like this:

```
id  value   type    length  dim
P01 TODO    TODO    TODO    TODO
```

The case file looks like this:

```
P01 = quote(c(TRUE, 2L, 3.5)),
```

The syntax map above explains `quote()`, `c()`, and the `L` suffix. The later cases also use `NA_real_` and `NA_integer_`. Those constants create missing values with a specified type. A plain `NA` is logical, and R coerces it to the common type required by the surrounding atomic vector.

Read the rules below and the notation after them, fill the table, write the rule behind each case in `REASONING.md`, then commit both. Only then run anything. The reasoning is the part this assignment grades most closely. A pair that copies the public answer table skips the work the reasoning asks for. The rubric reads the reasoning against the prediction commit.

The Rules You Need

An **atomic vector** stores one basic type. When `c()` combines logical, integer, double, or character values, R converts the whole vector to a common type. A list can keep unlike types in separate elements.

For the four atomic types in this assignment, R climbs a coercion ladder from logical to integer to double to character. It stops at the highest type present. `c(TRUE, 2L)` gives you integers. Add a `3.5` and the whole vector becomes double. Put a string anywhere in it and every element becomes character, including the logical value, which becomes `"TRUE"`.

R applies most arithmetic operators element by element. If one operand is shorter, R repeats it. This is **recycling**. In a 4-element vector plus a 2-element vector, the shorter operand contributes its first element to positions 1 and 3, then its second element to positions 2 and 4.

Recycling doesn't require the lengths to divide evenly. Add a 2-element vector to a 3-element one and R recycles anyway, reusing the first element a third time and printing a warning while it does. The result still comes back. A warning isn't an error, so a script that produces one keeps running and hands you an answer built on a partial repeat.

A matrix recycles the same way, but you have to know how a matrix is laid out first. R fills and stores one by columns, so `matrix(1:6, nrow = 2)` puts 1 and 2 in the first column, 3 and 4 in the second, 5 and 6 in the third. Underneath it's still the vector `1 2 3 4 5 6` with a `dim` attribute attached. Adding `c(10, 20)` recycles that pair down that same underlying order, so 10 goes to the first, third, and fifth stored values while 20 goes to the second, fourth, and sixth. In a two-row matrix that works out to adding 10 to every entry in the top row and 20 to every entry in the bottom one. Read the matrix by rows instead and you'll predict the additions backward.

R indices begin at 1. Positive indices select positions. Negative indices exclude positions. A logical index selects positions where the corresponding logical value is `TRUE`. Index 0 selects nothing and returns a zero-length result of the same atomic type.

The single bar in `a | b` computes logical OR at every position. `TRUE | NA` is `TRUE` because the known operand already determines the result. `FALSE | NA` is `NA`. This rule lets `is.na(x) | x > 10L` select a missing element even though comparing that element with `10L` produces `NA`.

`NA` is a missing value. Arithmetic with a missing operand usually produces a missing result. Comparisons do too. Functions such as `mean()` accept an `na.rm` argument when removing missing values is a meaningful choice.

`ifelse(test, yes, no)` makes one choice per position. It takes the value from `yes` where `test` is `TRUE` and from `no` where `test` is `FALSE`. A length-one `yes` or `no` value repeats across the required positions.

A factor stores integer codes and carries a set of level labels. A matrix and an array are atomic vectors with a `dim` attribute. A data frame is a list of equal-length columns with row and column dimensions.

Subassignment can change more than a value. If an integer vector receives a double, R widens the whole vector to double so the new value fits. That's a mixed-mode assignment. It retypes the existing elements along with the slot you wrote to.

Required Reading

The 16 cases grade those rules. This manual gives you a working map. A full R introduction would take more time than this assignment allows. Use the guide below as required reading while you work.

Read *An Introduction to R*, the official guide written by the R Core Team. It documents R 4.6.1, the same version the workflow runs, and it uses no packages, so nothing in it will send you installing things:

```
https://cran.r-project.org/doc/manuals/R-intro.html
```

Its sections line up with this problem set closely enough to read by topic:

What you're predicting	Where to read it
Atomic vectors and their one shared type	3.1, Intrinsic attributes: mode and length
Recycling in arithmetic	2.2, Vector arithmetic
Recycling a vector into a matrix	5.4.1, Mixed vector and array arithmetic
Indexing by position, exclusion, or logical value	2.7, Index vectors
Missing values	2.5, Missing values
What `class()` reports	3.4, The class of an object
Factors	Chapter 4, Ordered and unordered factors
Arrays and matrices	5.1, Arrays
Lists and data frames	6.1 and 6.3

The official guide talks about an object's **mode** and never mentions `typeof()` at all. The two aren't synonyms. They part ways on the objects you're predicting:

```
mode(c(1L, 2L))     # "numeric"
typeof(c(1L, 2L))   # "integer"
```

Half of the 16 cases have `integer` as their answer, so a prediction table filled in the guide's vocabulary loses eight checks. Read it for the rules and predict in `typeof()`'s words.

Advanced R, by Hadley Wickham, is the second reading. Chapter 3, "Vectors," covers coercion, attributes, factors, lists, and data frames in the same vocabulary this problem set uses, and its coercion section is the clearest short statement of the ladder you'll find:

```
https://adv-r.hadley.nz/vectors-chap.html
```

Skip its tibble material. A tibble is a data frame variant from an add-on package. This problem set uses base R data frames.

When you're offline or just want the exact behavior of one thing, R's own help is the fastest reference. At an R prompt, `?"["` explains indexing, `?factor` explains factors, and `?NA` explains missing values.

Grader Notation

The `value` field is one line of text describing an object. The grader compares it letter for letter against the description R produces, so the notation is part of the answer. Every form the 16 cases can produce appears below.

R value	Written as
`c(1, 2, 3.5)`	`[1,2,3.5]`
`c(first = 8L, third = 9L)`	`[first=8,third=9]`
`c("a", "b")`	`["a","b"]`
`NA` in any position	`NA`, with no quotes, as in `[NA,12]`
`integer(0)`	`[]`
`factor(c("red", "blue", "red"), levels = c("blue", "red"))`	`factor[red,blue,red];levels=[blue,red]`
`list(count = 2L, tag = "x")`	`list{count=[2],tag=["x"]}`
`data.frame(id = 1:2, passed = c(TRUE, FALSE))`	`data.frame[2x2]{id=[1,2],passed=[TRUE,FALSE]}`
`matrix(1:6, nrow = 2)`	`matrix[2x3](1,2,3,4,5,6)`
`array(1:8, dim = c(2L, 2L, 2L))`	`array[2x2x2](1,2,3,4,5,6,7,8)`

Three mechanical rules go with the table. Character values keep their quotes and everything else drops them, so a description tells the string `"1"` apart from the number `1`. Matrices and arrays list their values in storage order, meaning down the columns. And no field contains a space anywhere, so `[1, 2]` fails a check that `[1,2]` passes. A cell padded to line up your columns in an editor fails for the same reason.

The other three fields are shorter. Every case in this assignment has one of these `typeof()` results: `double`, `integer`, `character`, `logical`, or `list`. `length` is a number. `dim` is either `none` or the dimensions joined with `x`, as in `2x3`.

You can also read the rule in `scripts/support.R`, the code that writes these descriptions. It's short. The grader and `./run` both call it, so nothing else decides what a description looks like.

Predict, Then Commit

Replace every `TODO` in `predictions.tsv`. Keep the tab-separated layout and the case order. In `REASONING.md`, replace every placeholder line with one to three sentences that name the rule behind that case's value, type, length, and dimensions. The reasoning carries the grade because it shows the work behind the table. A correct table with no reasoning earns no more than a Fair prediction band.

Reason from the rules you just read, not from a guess at the printed result. Here is one entry for a case that isn't in the corpus:

```
## P17
The coercion ladder runs logical, integer, double, character. The second value
is a double, so the `5L` becomes `5` and the result is double. The value is
[5,0.5] with length two and no dimensions.
```

Then check the table's format:

```
./lint
```

`./lint` reads the form only. It never opens `tests/expected.tsv` or evaluates a case. You can run it before the prediction commit without revealing an answer. It catches mistakes that otherwise look like

wrong predictions: a padded cell, a space inside a value, a tab replaced with spaces, or a missing or reordered row. One trailing space on every line fails all 16 `dim` checks. The comparison output won't point at that space.

It reports each fault with its line and column:

```
line 2, column dim: the cell has leading or trailing whitespace. The
comparison is exact, so "none " is not the same answer as "none".
```

When it says the table is well formed and complete, commit the table and the reasoning together before running any case:

```
git add predictions.tsv REASONING.md
git commit -m "docs: predict R values and structure"
```

The commit records your prediction before you inspect a case or open `tests/expected.tsv`. After that commit, you may run:

```
./run P01
```

The runner prints one description:

```
case: P01
value: [1,2,3.5]
type: double
length: 3
dim: none
```

Compare the observation with your row. When they differ, correct the table. Add a short correction entry under that case in `REASONING.md`. Name what you predicted, what R printed, and the rule you misread. Here is one for the same case outside the corpus, if you had predicted the integer-linear result:

```
Correction: I predicted [5,0.5] and the type integer. R printed a double because
the `0.5` sits above `5L` on the ladder. I read `c()` as keeping the first type
instead of climbing to the highest one present.
```

Keep the prediction commit and put the correction in a later commit:

```
git add predictions.tsv REASONING.md
git commit -m "fix: account for atomic vector coercion"
```

That history is more useful than a table made perfect after the answers were already on screen. Git shows which entries you wrote before execution and which you added after.

Part 2: Implement the Score Transformation

The prediction cases culminate in this transformation:

```
scores <- c(7, NA_real_, 12, 4)
adjusted <- ifelse(is.na(scores), 0, scores * 2)
adjusted[adjusted > 15] <- 15
adjusted
```

You now implement a related operation in two ways. Both functions receive a numeric vector, including its `names` attribute when present. For each element:

1. Replace a missing score with 0.
2. Double every nonmissing score.
3. Cap the result at 20.
4. Preserve the vector's `names` attribute.

The snippet at the top of this part is prediction case P14's body. It caps at 15. Your transformation caps at 20. That's why the sample shows Linus at 20. His 12 doubles to 24 before the cap.

For the published sample:

```
c(Ada = 8, Grace = NA_real_, Linus = 12, Barbara = 4)
```

both functions return:

```
c(Ada = 16, Grace = 0, Linus = 20, Barbara = 8)
```

Implement `clean_scores_vector()` with vector expressions and subassignment. Don't use `for`, `while`, `repeat`, `Map`, `lapply`, `sapply`, or `vapply` in that function.

Implement `clean_scores_scalar()` with a `for` loop and one element at a time. Don't call `clean_scores_vector()` from it. The loop gives you an element-at-a-time evaluation model to compare against R's usual vector model. The word *scalar* describes that implementation strategy. R still represents each selected score as a length-one vector.

What the Model Checks Can't See

Two of the 76 checks look at how your cleaners work rather than what they return. `clean_scores_vector_model` reads the calls in the vector function and fails if any of them is a forbidden one. `clean_scores_scalar_model` rewrites each `for` loop in the scalar function so it counts its own passes, then runs the rewrite on a four-element vector. It wants the right answer and at least one pass per element. It also fails a direct call to `clean_scores_vector()`.

Neither check can prove which evaluation model you used. The reason is R's semantics showing through. Each of these gets past one of the checks:

1. A loop that runs once per element and does nothing, followed by a vector expression that computes the answer. The counter sees four passes. It can't see that they were idle.
2. A scalar function that inspects its own body while it runs, finds the counter the check added, and bumps it. A function body is an ordinary language object in R, so the counting code is open to the code it measures.
3. `delegate <- clean_scores_vector`, then `delegate(scores)`. A function is a value like any other. Binding it to a second name doesn't copy anything, and the check only looks for the first name.
4. A vector function that hands its work to a helper with a loop inside. The check reads one function body, and the helper's body lives somewhere else.

All four come from the same fact. Names bind to values at runtime, and a function is just another value. A check that runs in the same R session as your code is reasoning about code that can rename, rewrap, or rewrite whatever the check looks at. Moving the check into a separate process doesn't help either. The process that runs your function is still an R session your code controls. The only automatic fix left is a rigid template for both functions. That would reject most correct solutions.

So the two checks guard against honest slips. The rest is a rubric call. Your source gets read during grading, helpers included. A submission that games either check is graded on what its source does, not on what the checker printed, and a deliberate bypass falls under a Poor rubric band below. It's also more work than just writing the function itself properly.

Build the Data Frame

Implement:

```
build_roster(student_names, groups, scores)
```

It returns a data frame with these columns in order:

Column	Required value
`name`	`student_names` as character values
`group`	a factor whose levels follow first appearance in `groups`
`raw_score`	the supplied scores without names
`adjusted_score`	`clean_scores_vector(scores)` without names
`passed`	`TRUE` when the adjusted score is at least 12, without names

Set `stringsAsFactors = FALSE` when constructing the data frame. The `group` column is already an explicit factor.

"Levels follow first appearance" is a rule with mechanics behind it, so run them once before you rely on it:

```
groups <- c("red", "blue", "red", "blue")

levels(factor(groups))                              # "blue" "red"
levels(factor(groups, levels = unique(groups)))  # "red"  "blue"
```

`factor()` sorts its levels when you don't supply them. For these two labels, sorting inverts the required order. `unique()` returns the distinct values in the order they first appear, which is the order the column has to carry. The level order is what decides the row order of anything grouped by this column later. An incorrect order here first becomes visible in `summarize_roster`.

Summarize the Data Frame

Implement:

```
summarize_roster(roster)
```

It returns a list with these items in order:

Item	Required value
`rows`	the row count as an integer
`missing_raw`	the count of missing raw scores as an integer
`mean_adjusted`	the mean adjusted score
`passed`	names of students whose `passed` value is `TRUE`
`mean_by_group`	one adjusted-score mean per level of the `group` factor, named by that level, in level order

Use vector expressions to select rows and compute these values. For the published sample the factor's levels are `red` then `blue`, so `mean_by_group` comes out as `c(red = 18, blue = 4)`.

`mean_by_group` has to be a named vector of doubles with no dimensions. That rules out `tapply()`, which looks like the right tool and returns a one-dimensional array instead. The check compares with `identical()`. An array carrying a `dim` attribute isn't identical to a plain named vector even when both print the same two numbers. `split()` separates `adjusted_score` into one part per level of the `group` factor, in level order. `sapply()` or `vapply()` then computes one mean per part and returns a named vector with no dimensions. A `c()` with `red` and `blue` written by hand works only for the sample. The grader also runs a roster with other groups.

Follow the Missing Value

Grace's `NA` is the element that makes the two implementations look different, so follow that one element through each of them.

The vector version never mentions Grace. Three whole-vector operations run in order. Her element is carried along by each one:

Step	The operation on the whole vector	What happens to Grace's element
1	an `is.na()` mask used to subassign 0	`NA` becomes `0`
2	arithmetic across the whole vector	`0 * 2` is `0`
3	a comparison mask used to subassign the cap	`0 > 20` is false, so `0` stays

Each row is one expression. The mask in rows 1 and 3 is a logical vector the same length as `scores`, and putting that mask inside `[ ]` on the left of `<-` is the subassignment the constraint asks for. That's what "vector expressions and subassignment" means in practice.

The scalar version reaches Grace on its second pass. Nothing else in the vector takes part:

Pass	Element	What the loop body does
1	Ada, 8	present, so double to 16, which is under the cap
2	Grace, `NA`	missing, so store 0 and skip the doubling
3	Linus, 12	present, so double to 24, then cap at 20
4	Barbara, 4	present, so double to 8, which is under the cap

Both tables end at the same four numbers. What changes is how the code refers to Grace. The vector version applies three expressions to the full vector and never refers to position 2. Its first mask makes the missing-value decision. The scalar version makes that decision during the second loop iteration, at a line where a debugger could stop.

The loop has one trap the vector version avoids. Copy `scores` into a local double vector with `setNames(as.double(scores), names(scores))`, since a bare `as.double()` strips the `names` attribute. Write elements back with `out[[i]] <- value` and the `names` attribute stays attached. Accumulate into a fresh vector with `out <- c(out, value)` and the names come out missing, which fails the check even though all four numbers are right.

Part 3: Compare Vector and Scalar Evaluation

Replace the placeholder in `ANALYSIS.md` with 300 to 450 words written by both partners. The grader counts every space-separated token in the file, including headings. The placeholder fails the check. A total under 300 or over 450 also fails. Aim for the middle of the range so that one last edit keeps the file inside it. Trace one input, including its missing value, through both cleaning functions. Then answer these questions:

- Which operations in the vector version act on all elements at once?
- Where does the scalar loop make evaluation order visible?
- Give one concrete input where careless recycling would change the answer.
- Which implementation would you keep in an R program, and what tradeoff does that choice make?

Use the functions you wrote as evidence. A generic comparison of loops and vectors doesn't answer the prompt.

Running the Checks

Run one case while working on predictions:

```
./run P10
```

Run the whole public grader after every coherent change:

```
./check.sh
```

A complete submission ends with:

```
== result ==
76/76 checks passed
```

`76/76` is the complete automated result. The rubric separately assesses your analysis, prediction history, collaboration, and workflow runs.

Check the table's form at any time, which reveals no answers:

```
./lint
```

`./run` exits 0 when it prints the case, 64 when it's called with the wrong number of arguments, and 65 when the case id is unknown or the case faults. `./lint` exits 0 when the table is well formed and complete, and 1 otherwise. `./check.sh` lists the same form faults before it scores, under a `== predictions.tsv`

`form ==` heading. Read that heading first when it appears. A fault listed there identifies a typing mistake before the grader scores a prediction.

`check.sh` exits 0 only when all 76 checks pass. A runtime error in one of your functions becomes a labeled failed check while the script continues. Start at the first failure.

Malformed prediction tables stop the grader before it scores anything. The table has to keep its five columns and all 16 ids in order. When it doesn't, the script says so and quits. A run without a `== result ==` line points to the layout of `predictions.tsv`. A stray tab, a deleted row, or spaces in place of tabs can cause that failure.

The supplied workflow runs the same command on Ubuntu, macOS, and Windows with R 4.6.1. Push early enough to see all three jobs finish. A local green run is required. The three workflow jobs must also pass from a clean checkout.

Milestones for the Week

Release day, Tuesday, October 13, is for meeting your partner, setting up R, cloning the starter, and running the red baseline. Seven full workdays follow.

Day	Date	Work
1	Wed Oct 14	Read all 16 expressions together and start Chapters 2 and 3 of *An Introduction to R*
2	Thu Oct 15	Fill the table independently, compare answers, and commit before execution
3	Fri Oct 16	Run P01 through P08 and correct the rules you misunderstood
4	Sat Oct 17	Run P09 through P16 and finish the prediction checks
5	Sun Oct 18	Implement both cleaning functions and compare their outputs
6	Mon Oct 19	Build and summarize the roster, then write `ANALYSIS.md` together
7	Tue Oct 20	Run `./check.sh`, push, and read all three workflow jobs

Wednesday, October 21, is for final verification and submission.

Deliverables

Submit one public GitHub repository for the pair containing:

1. A completed `predictions.tsv`
2. A completed `REASONING.md`, committed with the predictions and updated with a correction entry under every case you missed
3. The four completed functions in `src/analysis.R`

4. A 300 to 450 word `ANALYSIS.md`
5. The unchanged public cases, expectations, runner, grader, `scripts/` helpers, and workflow
6. A green `.github/workflows/test.yml` run on the submitted commit
7. A `README.md` that identifies both members and their GitHub usernames, replacing the starter's own

Both members must commit. Use the syllabus's semantic commit prefixes. Commit history should show the prediction step, with the reasoning, before execution-based corrections and show each member's implementation or review work.

After the final push, open the repository's commit history on GitHub and copy the full hash of the commit you want graded. One designated member emails both items included in the body in this form:

```
Repository: https://github.com/<your-account>/cmsc-124-ps2-starter
Commit: <full commit hash copied from GitHub>
```

The submitted hash pins the version I grade. The green workflow run must belong to that commit. A later commit doesn't change the submitted version. CC the rest of the pair. Use this subject line, joining each person's entry with `&`:

```
[CMSC 124 Lec] PS2 Group: LastName1, Initials1 & LastName2, Initials2
```

A trio adds its third member in the same format.

Each member also emails `reflection.txt` individually. Identify the work you did, one rule your prediction missed, one part your partner explained, and what you learned. Use:

```
[CMSC 124 Lec] PS2: LastName, Initials
```

For example: `[CMSC 124 Lec] PS2 Group: Sanchez, SM & Jocsing, RA` and `[CMSC 124 Lec] PS2: Sanchez, SM`.

Common Pitfalls

- Reading a factor's printed labels and predicting `character`. `typeof()` reports its integer storage.
- Writing a value description with spaces in it. The comparison is exact, so `[1, 2]` is a wrong answer to a case whose result is `[1,2]`. Run `./lint` and it identifies the cell before a check ever scores it.
- Treating a data frame's length as its row count. A data frame is a list of columns, so length counts columns.
- Reading a matrix left to right by rows. R fills and stores matrices by columns unless told otherwise.
- Assuming arithmetic stops when the shorter vector ends. R recycles it.
- Comparing `NA` with a number and expecting `TRUE` or `FALSE`. The comparison is missing too.
- Using `mean()` on data containing `NA` without deciding whether to remove the missing values.
- Dropping the `names` attribute from the cleaning result.
- Implementing the scalar function by calling the vector function. The outputs would pass through the same evaluation path, leaving nothing to compare.
- Running the cases before committing predictions. The Git history would no longer establish which answers were predictions.
- Leaving `REASONING.md` for after the run. Entries written once the answers are on screen are corrections, not predictions, and the grade reads the reasoning against the prediction commit.
- Editing `tests/expected.tsv` until the checks pass. The published copy replaces it during grading.

Academic Honesty

Use the required reading, R's documentation, and course material to learn the language rules. You may also discuss those rules with your assigned partner. The prediction commit, later correction commits, implementation, and analysis must reflect your pair's own reasoning.

Write `REASONING.md` before you run anything and commit it with `predictions.tsv`. Reasoning written after the run is not a prediction, and Git shows when each entry appeared. A correction entry is the one exception. It belongs in a later commit.

Large language models may explain a language feature, but submitting generated predictions, reasoning, functions, or analysis as your own work is cheating. Each partner must be able to trace any case and explain either implementation without a tool's help. The public answer table means this part relies on an honor rule. Copying it into `predictions.tsv` before doing the work is still cheating even though the file is available to you. Copying it into `REASONING.md` as a restated answer, with no rule, earns no prediction band either.

Important Dates

Event	Date
Problem set released	Tuesday, October 13, 2026
Pairing sheet published	Tuesday, October 13, 2026
Submission deadline	Wednesday, October 21, 2026

Both lecture slots inside the work period, October 15 and October 19, are yours to work in. There's no meeting on either day.

Lecture deadlines are strict. The laboratory's prescribed timeline and booking system don't apply.

Grading Rubric

Criterion	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Predictions and Reasoning (20%)	Every case has a reasoning entry in the prediction commit that names the rule behind each predicted field. Every miss has a correction entry that names the misread rule. All 64 prediction checks pass by the final commit.	Every case has a reasoning entry in the prediction commit. A few entries restate the answer without a rule, or a few misses have no correction entry.	Many entries are generic or restate the answer, or many misses have no correction entry. A correct table whose reasoning is absent from the prediction commit earns no more than Fair.	`REASONING.md` is missing or empty, or the predictions and the reasoning first appear after execution with no prediction record.
Vector and Scalar Functions (20%)	Both cleaning functions return the required named vector. One uses vector operations and subassignment. The other uses a scalar loop.	Both return the correct values with a minor naming or implementation-model mistake.	One function works, or both work only for the published sample because missing values or the cap are mishandled.	Both functions are missing, fail on the published sample, or deliberately perform a bypass.
Structured Result (20%)	`build_roster()` produces the required data frame and factor levels. `summarize_roster()` returns all five values with correct types, names, and order.	Both functions work with one minor type, name, or order mismatch.	One structured function works, or both omit several required attributes.	Both functions are missing or return the wrong structures.
Analysis (20%)	The 300 to 450 word comparison traces the supplied missing value through both functions, explains recycling with a concrete input, and argues for one implementation using code from the submission.	All questions are answered, with a thinner trace or tradeoff.	The response describes vectors and loops generally but doesn't connect them to the submitted code or a concrete recycling case.	Missing, outside the word range, or unable to explain the two evaluation models.
Collaboration and Reproducibility (20%)	Both members make atomic semantic commits. `./check.sh` reports 76/76 locally and the submitted commit is green on Ubuntu, macOS, and Windows.	Both members contribute and the public checks pass, with one workflow or history weakness.	Contributions are markedly uneven, or the final clean-checkout workflow is red despite a mostly working local submission.	One member did the work, the grader was altered, or the submission can't run through `./check.sh`.